

01 — Architecture du projet (UpcycleConnect)

Projet : **UpcycleConnect** (dépôt `renova-systems-projet-annuel`) — plateforme d'économie circulaire (dons/ventes d'objets, conteneurs/casiers connectés, espace pro, back-office admin).

Stack technique (réel, vérifié dans le code)

Couche	Techno	Preuve (fichier)
Backend	Go (<code>net/http</code> standard, pas de framework)	<code>API/main.go</code> , <code>API/go.mod</code>
Base de données	MySQL 8	<code>docker-compose.yml</code> (<code>image: mysql:8.0</code>), <code>API/bdd/db.go</code>
Frontend	HTML / CSS / JS vanilla (pas de framework) servi par Nginx	<code>Frontend/*.html</code> , <code>Frontend/script/*.js</code> , <code>Frontend/nginx.conf</code>
Conteneurisation	Docker + Docker Compose	<code>docker-compose.yml</code> , <code>docker-compose-prod.yml</code> , <code>API/Dockerfile</code> , <code>Frontend/Dockerfile</code>
Auth	JWT (HS256)	<code>API/auth/jwt.go</code>
Paieement	Stripe (Checkout + Connect)	<code>API/admin/stripe.go</code> , <code>API/admin/webhook.go</code>
Notifications push	OneSignal	<code>API/admin/notifications.go</code>

Structure générale

```
renova-systems-projet-annuel/
├── API/                               ← BACKEND Go (port 8081)
│   ├── main.go                       ← point d'entrée : enregistre toutes les routes + ListenAndServe(":8081")
│   ├── go.mod / go.sum               ← dépendances Go
│   ├── Dockerfile                   ← build multi-stage du backend
│   ├── auth/
│   │   └── jwt.go                   ← génération/vérification JWT + middlewares de rôle
│   ├── bdd/
│   │   ├── db.go                   ← connexion MySQL (NewDB)
│   │   └── *Req.go                  ← requêtes par domaine (userReq, annonceReq, boxReq...)
│   ├── models/                     ← structs Go (Annonce, User, Evenement...)
│   ├── route/                       ← déclaration des routes HTTP (une fonction Routes* par domaine)
│   └── admin/                       ← handlers HTTP (la logique des endpoints) + upload + pdf + stripe + webhook
├── Frontend/                         ← FRONTEND statique (servi par Nginx)
│   ├── *.html                      ← pages (login, espClient, espPro, admin_*, salarie/*, annonceAll...)
│   ├── script/                     ← JS (config.js, login.js, dashClient.js, admin/*, salarie/*)
│   ├── style/                      ← CSS
│   ├── assets/                     ← logo, favicon (logoUpcycle.png / .ico)
│   ├── nginx.conf                  ← config Nginx (proxy /api, /uploads, URLs propres)
│   └── Dockerfile                  ← build du conteneur frontend (nginx)
├── db/
│   └── init.sql                    ← schéma + données MySQL (import auto au 1er démarrage Docker)
```

- └─ docker-compose.yml ← stack LOCALE (build local : uc_mysql / uc_backend / uc_frontend)
- └─ docker-compose-prod.yml ← stack VM/PROD (images Docker Hub amelbdj/upcycle-*)
- └─ .env.example ← modèle de variables d'environnement
- └─ pa2026.sql ← dump SQL de référence

Où se trouve quoi (chemins exacts)

Élément	Chemin
Frontend	Frontend/
Backend / API	API/ (entrée : API/main.go)
Config Docker (local)	docker-compose.yml
Config Docker (prod/VM)	docker-compose-prod.yml
Dockerfile backend	API/Dockerfile
Dockerfile frontend	Frontend/Dockerfile
Config Nginx	Frontend/nginx.conf
Base de données (schéma + seed)	db/init.sql (et pa2026.sql de référence)
Variables d'environnement	.env.example (à copier en .env), injectées via docker-compose*.yaml
Connexion DB (code)	API/bdd/db.go
Config API côté front	Frontend/script/config.js

Comment le front communique avec l'API

1. Frontend/script/config.js définit **API_BASE_URL** automatiquement :
2. **WAMP local** (localhost + URL contenant /Frontend/) → http://localhost:8081 (backend Go en direct)
3. **Docker / VM** (sinon) → origin + "/api" (ex : https://upcycleconnect.pro/api)
4. Tous les appels JS font fetch(\ \${API_BASE_URL}/...) avec un header Authorization: Bearer (token en localStorage).
5. En Docker/VM, Nginx (Frontend/nginx.conf) reçoit /api/... , réécrit ^/api/(.*)\$ /\$1 et **proxy_pass** vers http://backend:8081 .

```
Navigateur —fetch /api/admin/users→ Nginx (frontend:80)
└─ rewrite /api/(.*) → /$1 → proxy_pass http://backend:8081/admin/users → Go
```

Comment l'API communique avec la base

1. API/bdd/db.go → NewDB() ouvre la connexion MySQL avec le DSN user:pass@tcp(host:port)/pa2026?parseTime=true&charset=utf8mb4 .
2. Les identifiants viennent des **variables d'environnement** (DB_HOST , DB_PORT , DB_USER , DB_PASS , DB_NAME) ; en local, valeurs par défaut localhost/3306/root/root/pa2026 .
3. La variable globale bdd.Db (*sql.DB) est utilisée par tous les fichiers API/bdd/*Req.go via Db.Query(...) , Db.QueryRow(...) , Db.Exec(...) .
4. En Docker, docker-compose.yml fournit DB_HOST: mysql (nom du service MySQL dans le réseau uc_net).

Schéma d'ensemble

```
[Navigateur]
| HTML/JS statiques + fetch API (Bearer JWT)
▼
[Nginx frontend:80] — / , /login, /admin ... → fichiers .html (URLs propres)
|                  — /api/* → proxy → backend:8081
|                  — /uploads/* → proxy → backend:8081/uploads (images/PDF)
▼
[Go backend:8081] — auth JWT (middlewares) — handlers (API/admin/*) — bdd (API/bdd/*Req.go)
▼
[MySQL mysql:3306] base `pa2026`
```